

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Пермский национальный исследовательский политехнический
университет»

Электротехнический факультет

Кафедра: Информационные технологии и автоматизированные системы
(ИТАС)

Направление подготовки: 09.03.04 – «Программная инженерия»

ОТЧЕТ

Лабораторная работа №11.

Тема: «Информационные динамические структуры»

По дисциплине «Основы алгоритмизации и программирования»

Вариант 23

Выполнила: студентка группы РИС-25-26

Лаптева Елена Владимировна

Принял: доц. Полякова О.А.

Пермь, 2026

Односвязные списки

Постановка задачи

char - узлы списка

- 1 - добавление элемента в конец списка (head)
- 2 - добавление элемента в конец списка (head + tail)
- 3 - добавление элемента в начало списка
- 4 - добавление элемента после указанного индекса в списке
- 5 - добавление k элементов после элемента с заданным ключом
- 6 - удаление элемента из начала списка
- 7 - удаление элемента из конца списка
- 8 - удаление элемента после указанного индекса в списке
- 9 - удаление элемента с заданным ключом
- 10 - показ списка
- 11 - запись списка в файл
- 12 - уничтожение списка
- 13 - восстановление списка из файла

Анализ задачи

Добавление в начало (addToBegin):

- создать новый узел;
- установить его next на текущий head;
- обновить head на новый узел.

Добавление в конец (pushBack):

- если список пуст — новый узел становится head;
- иначе пройти до последнего узла и установить его next на новый узел.

Вставка после индекса (insertAfterIndex):

- найти узел с заданным индексом;
- вставить новый узел между ним и следующим.

Удаление из начала (popFront):

- сохранить текущий head;
- переместить head на следующий узел;
- освободить память удалённого узла.

Удаление из конца (popBack):

- найти предпоследний узел;
- удалить последний узел;
- установить next предпоследнего узла в nullptr.

Удаление по индексу (removeAfterIndex):

- найти узел с указанным индексом;
- удалить следующий за ним узел;
- перенаправить указатель текущего узла на узел после удалённого.

Удаление по ключу (removeByKey):

- найти узел с заданным ключом;
- если узел — head, обновить head;
- иначе найти предыдущий узел и перенаправить его next;
- освободить память удалённого узла.

Поиск по ключу (findNodeByKey):

- пройти по списку от head, сравнивая data каждого узла с ключом;

- вернуть найденный узел или nullptr, если не найден.
- Вставка k элементов после ключа (insertKElementsAfterKey):
- найти узел с заданным ключом;
 - последовательно вставить k новых узлов после него.
-

Алгоритмы работы с файлами

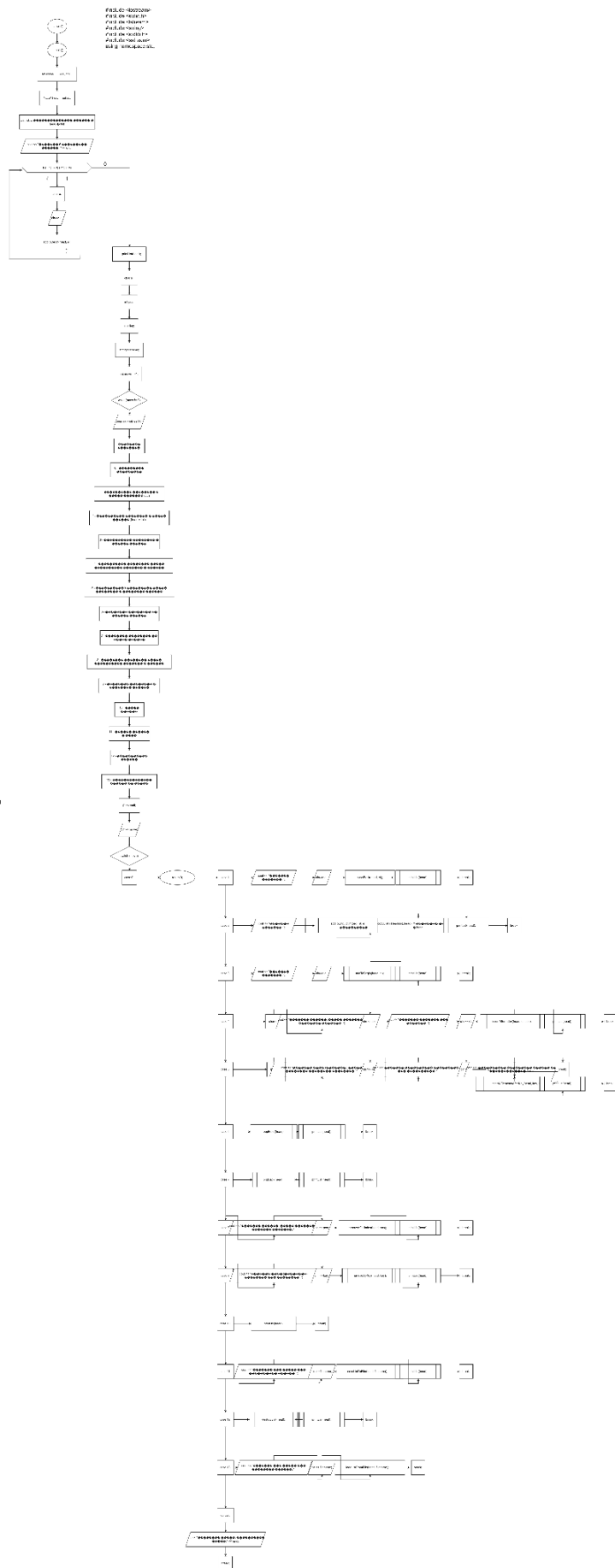
Сохранение в файл (saveListToFile):

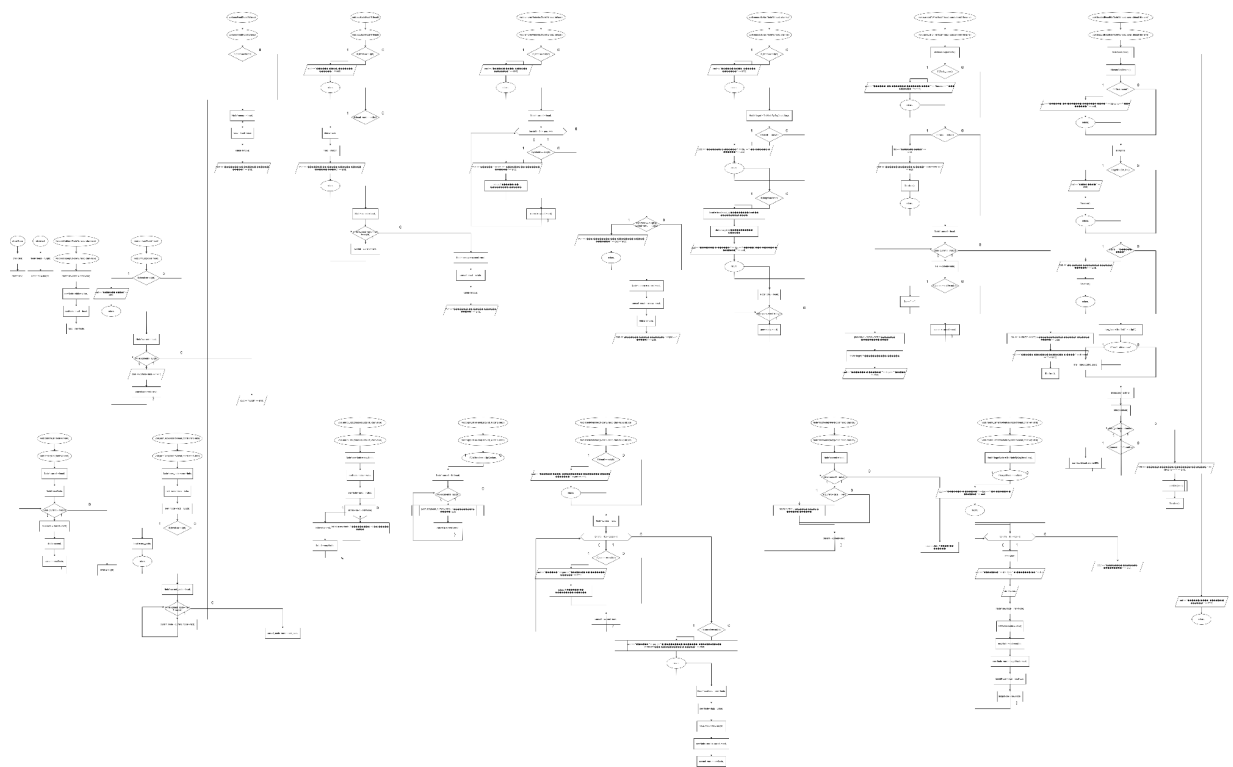
- открыть файл для записи;
- если список пуст, записать «Список пуст»;
- иначе пройти по списку, записывая data узлов через «->»;
- в конце добавить «-> nullptr»;
- закрыть файл.

Загрузка из файла (loadListFromFile):

- уничтожить текущий список;
- открыть файл для чтения;
- прочитать строку;
- если строка — «Список пуст», завершить;
- удалить окончание «-> nullptr» из строки;
- разобрать строку на элементы (пропуская «->»);
- добавить каждый элемент в новый список;
- вывести результат.

Блок-схема





Код

```
#include <iostream>
#include <stdio.h>
#include <fstream>
#include <string>
#include <stdlib.h>
#include <sstream>
using namespace std;

// Структура узла списка
struct Node {
    char data;
    Node* next;
};

// Структура списка с head и tail для быстрого доступа
struct List {
    Node* head = nullptr;
    Node* tail = nullptr;
};

// Функция добавления элемента в начало списка (для обычного head)
void addToBegin(Node*& head, char value) {
    Node* newNode = new Node;
    newNode->data = value;
    newNode->next = head;
    head = newNode;
}

// Функция вывода списка
void printList(Node* head) {
    if (head == nullptr) {
        cout << "Список пусм" << endl;
        return;
    }
}

Node* current = head;
```

```

while (current != nullptr) {
    cout << current->data << " -> ";
    current = current->next;
}
cout << "nullptr" << endl;
}

// Функция уничтожения списка
void destroyList(Node*& head) {
    Node* current = head;
    Node* nextNode;

    while (current != nullptr) {
        nextNode = current->next;
        delete current;
        current = nextNode;
    }
    head = nullptr;
}

// Функция добавления элемента в конец списка (хранение только head)
void pushBack(Node*& head, const char& data) {
    Node* new_node = new Node;
    new_node->data = data;
    new_node->next = nullptr;

    if (head == nullptr) {
        head = new_node;
        return;
    }

    Node* current_node = head;
    while (current_node->next != nullptr) {
        current_node = current_node->next;
    }
    current_node->next = new_node;
}

// Функция добавления элемента в начало через List (оптимизированная для case 3)
void addToEndOptimized(List& list, char value) {
    Node* newNode = new Node;
    newNode->data = value;
    newNode->next = nullptr;

    if (list.head == nullptr) {
        list.head = newNode;
        list.tail = newNode;
    }
    else {
        list.tail->next = newNode;
        list.tail = newNode; // Обновляем tail на новый узел
    }
}

// Функция копирования из List в обычный head
void copyListToHead(List& list, Node*& head) {
    if (list.head == nullptr) return;

    Node* current = list.head;
    while (current != nullptr) {
        pushBack(head, current->data); // Добавляем в конец head
        current = current->next;
    }
}

```

```

}

// Функция добавления после индекса
void insertAfterIndex(Node*& head, char value, int pos) {
    // Если список пуст
    if (head == nullptr) {
        cout << "Список пуст, невозможно вставить после индекса " << pos << endl;
        return;
    }

    // Находим узел с индексом pos
    Node* current = head;
    for (int i = 0; i < pos; i++) {
        if (current == nullptr) {
            cout << "Индекс " << pos << " выходит за пределы списка" << endl;
            return; // Индекс за пределами списка
        }
        current = current->next;
    }

    // Если дошли до конца списка (current == nullptr)
    if (current == nullptr) {
        cout << "Индекс " << pos << " — последний элемент. Используйте pushBack для добавления в конец."
        << endl;
        return;
    }

    // Создаём новый узел
    Node* newNode = new Node;
    newNode->data = value;
    newNode->next = nullptr;

    // Вставляем новый узел после current
    newNode->next = current->next;
    current->next = newNode;
}

// Функция поиска узла по ключу
Node* findNodeByKey(Node* head, char key) {
    Node* current = head;
    while (current != nullptr) {
        if (current->data == key) {
            return current; // Найден узел с нужным ключом
        }
        current = current->next;
    }
    return nullptr; // Ключ не найден
}

// Функция вставки k элементов
void insertKElementsAfterKey(Node*& head, char key, int k) {
    // Находим узел с заданным ключом
    Node* targetNode = findNodeByKey(head, key);

    if (targetNode == nullptr) {
        cout << "Элемент с ключом '" << key << "' не найден в списке." << endl;
        return;
    }

    // Добавляем k элементов после targetNode
    for (int i = 0; i < k; i++) {
        char value;
        cout << "Введите " << (i + 1) << "-й элемент из " << k << ": ";
    }
}

```

```

    cin >> value;

    // Создаём новый узел
    Node* newNode = new Node;
    newNode->data = value;
    newNode->next = nullptr;

    // Вставляем новый узел после targetNode
    newNode->next = targetNode->next;
    targetNode->next = newNode;

    // Обновляем targetNode на только что добавленный узел
    // (чтобы следующий элемент добавился после него)
    targetNode = newNode;
}
cout << "Элементы успешно добавлены." << endl;
}

// Функция удаления из начала
void popFront(Node*& head) {
    // Если список пуст — нечего удалять
    if (head == nullptr) {
        cout << "Список пуст, удалять нечего." << endl;
        return;
    }

    // Сохраняем указатель на удаляемый узел (текущий head)
    Node* remove = head;

    // Обновляем head — теперь это следующий узел
    head = head->next;

    // Освобождаем память удаляемого узла
    delete remove;

    cout << "Элемент из начала списка удалён." << endl;
}

// Функция удаления из конца
void popBack(Node*& head) {
    // Если список пуст
    if (head == nullptr) {
        cout << "Список пуст, удалять нечего." << endl;
        return;
    }

    // Если в списке только один элемент
    if (head->next == nullptr) {
        delete head;
        head = nullptr;
        cout << "Элемент из конца списка удалён (список пуст)." << endl;
        return;
    }

    // Находим предпоследний узел
    Node* current = head;
    while (current->next->next != nullptr) {
        current = current->next;
    }

    // Удаляем последний узел
    Node* remove = current->next;
    current->next = nullptr;
}

```



```

delete remove;

cout << "Элемент из конца списка удалён." << endl;
}

// удаление по индексу
void removeAfterIndex(Node*& head, int pos) {
    // Если список пуст
    if (head == nullptr) {
        cout << "Список пуст, нечего удалять." << endl;
        return;
    }

    // Находим узел с индексом pos (узел, после которого нужно удалить элемент)
    Node* current = head;
    for (int i = 0; i < pos; i++) {
        if (current == nullptr) {
            cout << "Индекс " << pos << " выходит за пределы списка." << endl;
            return; // Индекс за пределами списка
        }
        current = current->next;
    }

    // Если дошли до конца списка (current == nullptr) или следующий узел отсутствует
    if (current == nullptr || current->next == nullptr) {
        cout << "Нет элемента для удаления после индекса " << pos << endl;
        return;
    }

    // Узел, который нужно удалить
    Node* remove = current->next;

    // Переключаем указатель текущего узла на узел после удаляемого
    current->next = remove->next;

    // Освобождаем память удаляемого узла
    delete remove;

    cout << "Элемент после индекса " << pos << " удалён." << endl;
}

// удаление по ключу
void removeByKey(Node*& head, char key) {
    // Если список пуст
    if (head == nullptr) {
        cout << "Список пуст, нечего удалять." << endl;
        return;
    }

    // Ищем узел с заданным ключом
    Node* target = findNodeByKey(head, key);

    // Если узел не найден
    if (target == nullptr) {
        cout << "Элемент с ключом '" << key << "' не найден в списке." << endl;
        return;
    }

    // Случай 1: удаляемый узел — первый в списке (head)
    if (target == head) {
        head = head->next; // Обновляем head на следующий узел
        delete target;      // Освобождаем память
        cout << "Элемент с ключом '" << key << "' удалён (был первым в списке)." << endl;
    }
}

```

```

    return;
}

// Случай 2: удаляемый узел не первый — нужно найти предыдущий узел
Node* prev = head;
while (prev->next != target) {
    prev = prev->next;
}

// Удаляем найденный узел
prev->next = target->next; // Обходим удаляемый узел
delete target;           // Освобождаем память
cout << "Элемент с ключом '" << key << "' удалён." << endl;
}

// запись в файл
void saveListToFile(Node* head, const string& filename) {
    ofstream file(filename);
    if (!file.is_open()) {
        cout << "Ошибка: не удалось открыть файл '" << filename << "' для записи." << endl;
        return;
    }

    if (head == nullptr) {
        file << "Список пуст" << endl;
        cout << "Список записан в файл '" << filename << "' << endl;
        file.close();
        return;
    }

    Node* current = head;
    while (current != nullptr) {
        file << current->data;
        if (current->next != nullptr) {
            file << " -> ";
        }
        current = current->next;
    }
    file << " -> nullptr" << endl; // Исправленный формат: пробел перед -> nullptr

    cout << "Список успешно записан в файл '" << filename << "' << endl;
    file.close();
}

// чтение из файла
void loadListFromFile(Node*& head, const string& filename) {
    // Сначала уничтожаем текущий список
    destroyList(head);

    ifstream file(filename);
    if (!file.is_open()) {
        cout << "Ошибка: не удалось открыть файл '" << filename << "' для чтения." << endl;
        return;
    }

    string line;
    if (!getline(file, line)) {
        cout << "Файл пуст." << endl;
        file.close();
        return;
    }

    // Проверяем, пуст ли список

```

```

if (line == "Список пуст") {
    cout << "Из файла загружен пустой список." << endl;
    file.close();
    return;
}

// Удаляем "-> nullptr" в конце строки
size_t pos = line.find("-> nullptr");
if (pos != string::npos) {
    line = line.substr(0, pos);
}

// Разбираем строку на элементы
stringstream ss(line);
string element;

while (getline(ss, element, ' ')) {
    // Пропускаем стрелки "->"
    if (element == "->") continue;

    // Добавляем элемент в список
    pushBack(head, element[0]);
}

cout << "Список успешно загружен из файла '" << filename << "' << endl;
printList(head);
file.close();
}

int main() {
    setlocale(LC_ALL, "ru");

    Node* head = nullptr;
    List list; // Инициализируем список с head и tail

    cout << "Введите 5 элементов списка: " << endl;
    for (int i = 0; i < 5; i++) {
        char a;
        cin >> a;
        addToBegin(head, a);
    }

    printList(head);

    char el;
    int pos;
    char key;
    string filename;
    int move = -1;
    while (move != 0) {
        cout << endl << R"(
Выберите действие:
0 - завершить программу
1 - добавление элемента в конец списка (head)
2 - добавление элемента в конец списка (head + tail)
3 - добавление элемента в начало списка
4 - добавление элемента после указанного индекса в списке
5 - добавление k элементов после элемента с заданным ключом
6 - удаление элемента из начала списка
7 - удаление элемента из конца списка
8 - удаление элемента после указанного индекса в списке
9 - удаление элемента с заданным ключом
10 - показ списка

```

```

11 - запись списка в файл
12 - уничтожение списка
13 - восстановление списка из файла
)" << endl;
    cin >> move;

    switch (move) {
    case 0:
        return 0;

    case 1:
        cout << "Введите элемент: ";
        cin >> el;
        pushBack(head, el);
        printList(head);
        break;

    case 2:
        cout << "Введите элемент: ";
        cin >> el;
        addToEndOptimized(list, el); // Добавляем в конец оптимизированного списка
        copyListToHead(list, head); // Копируем обратно в head
        printList(head);
        break;

    case 3:
        cout << "Введите элемент: ";
        cin >> el;
        addToBegin(head, el);
        printList(head);
        break;

    case 4:
        char el;
        cout << "Введите индекс, после которого вставить элемент: ";
        cin >> pos;
        cout << "Введите элемент для вставки: ";
        cin >> el;
        insertAfterIndex(head, el, pos);
        printList(head);
        break;

    case 5:
        int k;
        cout << "Введите ключ (элемент), после которого добавить элементы: ";
        cin >> key;
        cout << "Введите количество элементов (k) для добавления: ";
        cin >> k;
        if (k <= 0) {
            cout << "Количество элементов должно быть положительным." << endl;
            break;
        }
        insertKElementsAfterKey(head, key, k);
        printList(head);
        break;

    case 6:
        popFront(head);
        printList(head);
        break;

    case 7:
        popBack(head);

```

```

    printList(head);
    break;

case 8:
    cout << "Введите индекс, после которого удалить элемент: ";
    cin >> pos;
    removeAfterIndex(head, pos);
    printList(head);
    break;

case 9:
    cout << "Введите ключ (значение) элемента для удаления: ";
    cin >> key;
    removeByKey(head, key);
    printList(head);
    break;

case 10:
    printList(head);
    break;

case 11:
    cout << "Введите имя файла для сохранения списка: ";
    cin >> filename;
    saveListToFile(head, filename);
    printList(head);
    break;

case 12:
    destroyList(head);
    printList(head);
    break;

case 13:
    cout << "Введите имя файла для загрузки списка: ";
    cin >> filename;
    loadListFromFile(head, filename);
    break;

default:
    cout << "Неверный выбор. Попробуйте снова." << endl;
    break;
}
}
}

```

Результат выполнения программы

Введите 5 элементов списка:

```
w
e
4
g
2
2 -> g -> 4 -> e -> w -> nullptr
```

Выберите действие:

```
0 - завершить программу
1 - добавление элемента в конец списка (head)
2 - добавление элемента в конец списка (head + tail)
3 - добавление элемента в начало списка
4 - добавление элемента после указанного индекса в списке
5 - добавление k элементов после элемента с заданным ключом
6 - удаление элемента из начала списка
7 - удаление элемента из конца списка
8 - удаление элемента после указанного индекса в списке
9 - удаление элемента с заданным ключом
10 - показ списка
11 - запись списка в файл
12 - уничтожение списка
13 - восстановление списка из файла
```

1

Введите элемент: 1

```
2 -> g -> 4 -> e -> w -> 1 -> nullptr
```

2

Введите элемент: 3

```
2 -> g -> 4 -> e -> w -> 1 -> 3 -> nullptr
```

3

Введите элемент: 1

```
1 -> 2 -> g -> 4 -> e -> w -> 1 -> 3 -> nullptr
```

4

Введите индекс, после которого вставить элемент: 2

Введите элемент для вставки: 5

```
1 -> 2 -> g -> 5 -> 4 -> e -> w -> 1 -> 3 -> nullptr
```

5

Введите ключ (элемент), после которого добавить элементы: 5

Введите количество элементов (k) для добавления: 3

Введите 1-й элемент из 3: g

Введите 2-й элемент из 3: r

Введите 3-й элемент из 3: t

Элементы успешно добавлены.

1 -> 2 -> g -> 5 -> g -> r -> t -> 4 -> e -> w -> 1 -> 3 -> nullptr

6

Элемент из начала списка удалён.

2 -> g -> 5 -> g -> r -> t -> 4 -> e -> w -> 1 -> 3 -> nullptr

7

Элемент из конца списка удалён.

2 -> g -> 5 -> g -> r -> t -> 4 -> e -> w -> 1 -> nullptr

8

Введите индекс, после которого удалить элемент: 3

Элемент после индекса 3 удалён.

2 -> g -> 5 -> g -> t -> 4 -> e -> w -> 1 -> nullptr

9

Введите ключ (значение) элемента для удаления: 2

Элемент с ключом '2' удалён (был первым в списке).

g -> 5 -> g -> t -> 4 -> e -> w -> 1 -> nullptr

10

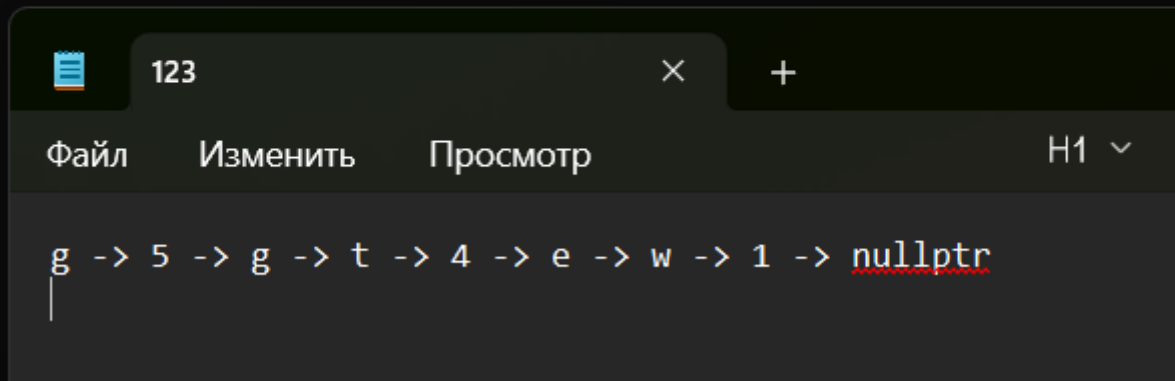
g -> 5 -> g -> t -> 4 -> e -> w -> 1 -> nullptr

11

Введите имя файла для сохранения списка: 123

Список успешно записан в файл '123'

g -> 5 -> g -> t -> 4 -> e -> w -> 1 -> nullptr



12

Список пуст

13

Введите имя файла для загрузки списка: 123

Список успешно загружен из файла '123'

g -> 5 -> g -> t -> 4 -> e -> w -> 1 -> nullptr

0

C:\Users\elenalapteva\source\repos\todom 0 (0x0).

Чтобы автоматически закрывать консоль
томатически закрыть консоль при оста
Нажмите любую клавишу, чтобы закрыть

Двусвязные списки

Постановка задачи

char - узлы списка

- 1 - добавление элемента в конец списка (head)
- 2 - добавление элемента в конец списка (head + tail)
- 3 - добавление элемента в начало списка
- 4 - добавление элемента после указанного индекса в списке
- 5 - добавление k элементов после элемента с заданным ключом
- 6 - удаление элемента из начала списка
- 7 - удаление элемента из конца списка
- 8 - удаление элемента после указанного индекса в списке
- 9 - удаление элемента с заданным ключом
- 10 - показ списка
- 11 - запись списка в файл
- 12 - уничтожение списка
- 13 - восстановление списка из файла

Анализ задачи

Добавление в начало (addToBegin):

- создать новый узел;
- установить его next на текущий head;
- обновить head на новый узел.

Добавление в конец (pushBack):

- если список пуст — новый узел становится head;
- иначе пройти до последнего узла и установить его next на новый узел.

Вставка после индекса (insertAfterIndex):

- найти узел с заданным индексом;
- вставить новый узел между ним и следующим.

Удаление из начала (popFront):

- сохранить текущий head;
- переместить head на следующий узел;
- освободить память удалённого узла.

Удаление из конца (popBack):

- найти предпоследний узел;
- удалить последний узел;
- установить next предпоследнего узла в nullptr.

Удаление по индексу (removeAfterIndex):

- найти узел с указанным индексом;
- удалить следующий за ним узел;
- перенаправить указатель текущего узла на узел после удалённого.

Удаление по ключу (removeByKey):

- найти узел с заданным ключом;
- если узел — head, обновить head;
- иначе найти предыдущий узел и перенаправить его next;
- освободить память удалённого узла.

Поиск по ключу (findNodeByKey):

- пройти по списку от head, сравнивая data каждого узла с ключом;

- вернуть найденный узел или nullptr, если не найден.
- Вставка k элементов после ключа (insertKElementsAfterKey):
- найти узел с заданным ключом;
 - последовательно вставить k новых узлов после него.
-

Алгоритмы работы с файлами

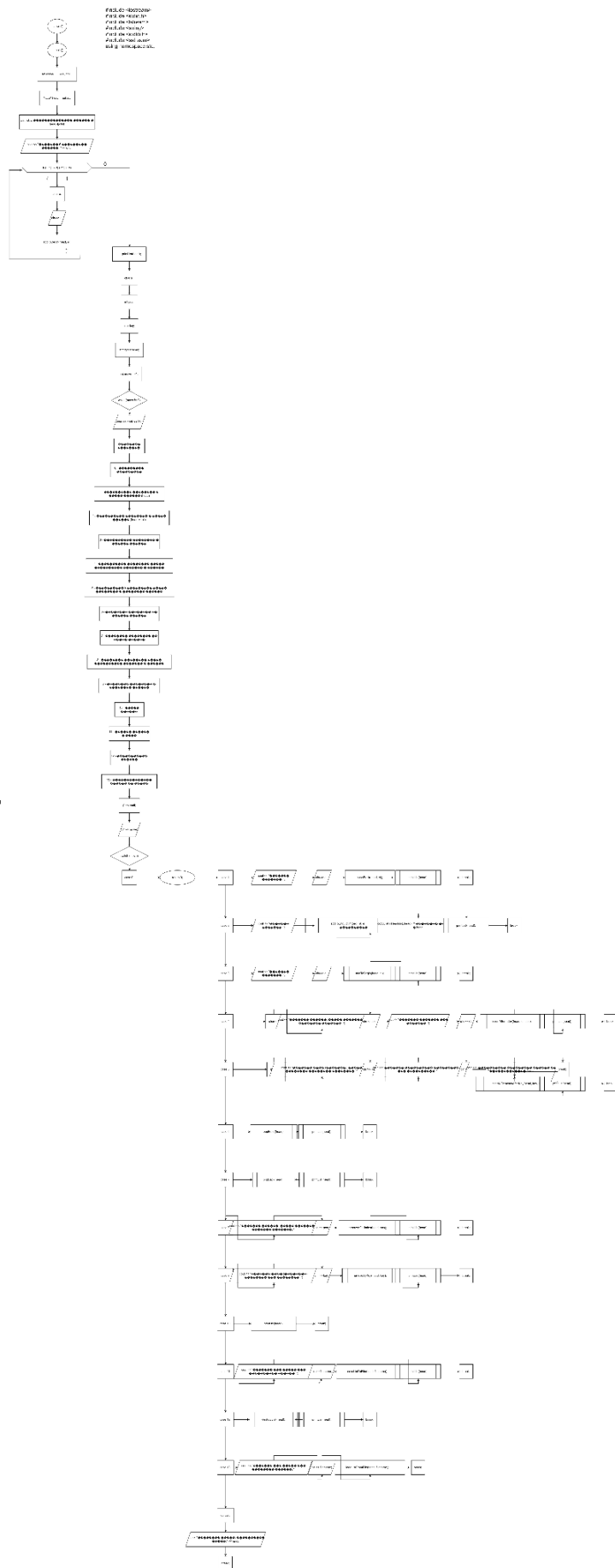
Сохранение в файл (saveListToFile):

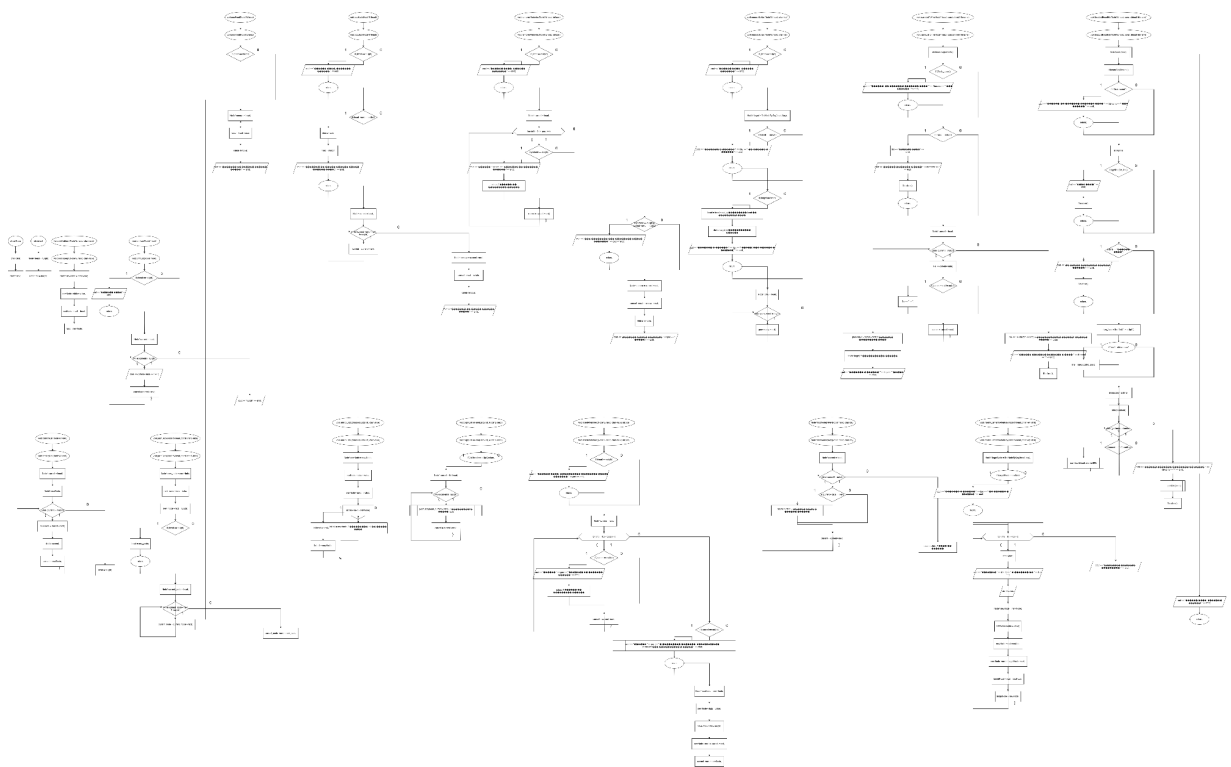
- открыть файл для записи;
- если список пуст, записать «Список пуст»;
- иначе пройти по списку, записывая data узлов через «->»;
- в конце добавить «-> nullptr»;
- закрыть файл.

Загрузка из файла (loadListFromFile):

- уничтожить текущий список;
- открыть файл для чтения;
- прочитать строку;
- если строка — «Список пуст», завершить;
- удалить окончание «-> nullptr» из строки;
- разобрать строку на элементы (пропуская «->»);
- добавить каждый элемент в новый список;
- вывести результат.

Блок-схема





Код

```
#include <iostream>
#include <stdio.h>
#include <fstream>
#include <string>
#include <stdlib.h>
#include <sstream>
using namespace std;
```

// Структура узла двусвязного списка

```
struct Node {
    char data;
    Node* next = nullptr;
    Node* prev = nullptr;
};
```

// Структура списка с head и tail для быстрого доступа

```
struct List {
    Node* head = nullptr;
    Node* tail = nullptr;
};
```

// Функция добавления элемента в начало списка (для обычного head)

```
void addToBegin(Node*& head, char value) {
    Node* newNode = new Node;
    newNode->data = value;
    newNode->next = head;
    newNode->prev = nullptr;
```

```
    if (head != nullptr) {
        head->prev = newNode;
    }
```

```

    }
    head = newNode;
}

// Функция вывода списка
void printList(Node* head) {
    if (head == nullptr) {
        cout << "Список пуст" << endl;
        return;
    }

    Node* current = head;
    while (current != nullptr) {
        cout << current->data << " -> ";
        current = current->next;
    }
    cout << "nullptr" << endl;
}

// Функция уничтожения списка
void destroyList(Node*& head) {
    Node* current = head;
    Node* nextNode;

    while (current != nullptr) {
        nextNode = current->next;
        delete current;
        current = nextNode;
    }
    head = nullptr;
}

// Функция добавления элемента в конец списка (хранение только head)
void pushBack(Node*& head, const char& data) {
    Node* new_node = new Node;
    new_node->data = data;
    new_node->next = nullptr;
    new_node->prev = nullptr;

    if (head == nullptr) {
        head = new_node;
        return;
    }

    Node* current_node = head;
    while (current_node->next != nullptr) {
        current_node = current_node->next;
    }
    current_node->next = new_node;
    new_node->prev = current_node;
}

```

// Функция добавления элемента в конец через List (оптимизированная для case 3)

```
void addToEndOptimized(List& list, char value) {
```

```
    Node* newNode = new Node;
```

```
    newNode->data = value;
```

```
    newNode->next = nullptr;
```

```
    newNode->prev = nullptr;
```

```
    if (list.head == nullptr) {
```

```
        list.head = newNode;
```

```
        list.tail = newNode;
```

```
    }
```

```
    else {
```

```
        list.tail->next = newNode;
```

```
        newNode->prev = list.tail;
```

```
        list.tail = newNode; // Обновляем tail на новый узел
```

```
    }
```

```
}
```

// Функция копирования из List в обычный head

```
void copyListToHead(List& list, Node*& head) {
```

```
    if (list.head == nullptr) return;
```

```
    Node* current = list.head;
```

```
    while (current != nullptr) {
```

```
        pushBack(head, current->data); // Добавляем в конец head
```

```
        current = current->next;
```

```
    }
```

```
}
```

// Функция добавления после индекса

```
void insertAfterIndex(Node*& head, char value, int pos) {
```

```
    // Если список пуст
```

```
    if (head == nullptr) {
```

```
        cout << "Список пуст, невозможно вставить после индекса " << pos << endl;
```

```
        return;
```

```
    }
```

```
    // Находим узел с индексом pos
```

```
    Node* current = head;
```

```
    for (int i = 0; i < pos; i++) {
```

```
        if (current == nullptr) {
```

```
            cout << "Индекс " << pos << " выходит за пределы списка" << endl;
```

```
            return; // Индекс за пределами списка
```

```
        }
```

```
        current = current->next;
```

```
    }
```

```
    // Если дошли до конца списка (current == nullptr)
```

```
    if (current == nullptr) {
```

```
        cout << "Индекс " << pos << " — последний элемент. Используйте pushBack для добавления в конец." << endl;
```

```
        return;
```

```

}

// Создаём новый узел
Node* newNode = new Node;
newNode->data = value;
newNode->next = nullptr;
newNode->prev = nullptr;

// Вставляем новый узел после current
newNode->next = current->next;
newNode->prev = current;

if (current->next != nullptr) {
    current->next->prev = newNode;
}
current->next = newNode;
}

// Функция поиска узла по ключу
Node* findNodeByKey(Node* head, char key) {
    Node* current = head;
    while (current != nullptr) {
        if (current->data == key) {
            return current; // Найден узел с нужным ключом
        }
        current = current->next;
    }
    return nullptr; // Ключ не найден
}

// Функция вставки k элементов
void insertKElementsAfterKey(Node*& head, char key, int k) {
    // Находим узел с заданным ключом
    Node* targetNode = findNodeByKey(head, key);

    if (targetNode == nullptr) {
        cout << "Элемент с ключом '" << key << "' не найден в списке." << endl;
        return;
    }

    // Добавляем k элементов после targetNode
    for (int i = 0; i < k; i++) {
        char value;
        cout << "Введите " << (i + 1) << "-й элемент из " << k << ": ";
        cin >> value;

        // Создаём новый узел
        Node* newNode = new Node;
        newNode->data = value;
        newNode->next = nullptr;
        newNode->prev = nullptr;
    }
}

```

```

// Вставляем новый узел после targetNode
newNode->next = targetNode->next;
newNode->prev = targetNode;

if (targetNode->next != nullptr) {
    targetNode->next->prev = newNode;
}
targetNode->next = newNode;

// Обновляем targetNode на только что добавленный узел
// (чтобы следующий элемент добавился после него)
targetNode = newNode;
}
cout << "Элементы успешно добавлены." << endl;
}

// Функция удаления из начала
void popFront(Node*& head) {
    // Если список пуст — нечего удалять
    if (head == nullptr) {
        cout << "Список пуст, удалять нечего." << endl;
        return;
    }

    // Сохраняем указатель на удаляемый узел (текущий head)
    Node* remove = head;

    // Обновляем head — теперь это следующий узел
    head = head->next;

    if (head != nullptr) {
        head->prev = nullptr;
    }

    // Освобождаем память удаляемого узла
    delete remove;

    cout << "Элемент из начала списка удалён." << endl;
}

// Функция удаления из конца
void popBack(Node*& head) {
    // Если список пуст
    if (head == nullptr) {
        cout << "Список пуст, удалять нечего." << endl;
        return;
    }

    // Если в списке только один элемент
    if (head->next == nullptr) {
        delete head;
        head = nullptr;
    }
}

```



```

    cout << "Элемент из конца списка удалён (список пуст)." << endl;
    return;
}

// Находим последний узел
Node* current = head;
while (current->next != nullptr) {
    current = current->next;
}

// Удаляем последний узел
if (current->prev != nullptr) {
    current->prev->next = nullptr;
}
delete current;

cout << "Элемент из конца списка удалён." << endl;
}

// удаление по индексу
void removeAfterIndex(Node*& head, int pos) {
    // Если список пуст
    if (head == nullptr) {
        cout << "Список пуст, нечего удалять." << endl;
        return;
    }

    // Находим узел с индексом pos (узел, после которого нужно удалить элемент)
    Node* current = head;
    for (int i = 0; i < pos; i++) {
        if (current == nullptr) {
            cout << "Индекс " << pos << " выходит за пределы списка." << endl;
            return; // Индекс за пределами списка
        }
        current = current->next;
    }

    // Если дошли до конца списка (current == nullptr) или следующий узел отсутствует
    if (current == nullptr || current->next == nullptr) {
        cout << "Нет элемента для удаления после индекса " << pos << endl;
        return;
    }

    // Узел, который нужно удалить
    Node* remove = current->next;

    // Переключаем указатель текущего узла на узел после удаляемого
    current->next = remove->next;

    if (remove->next != nullptr) {
        remove->next->prev = current;
    }
}

```

```

// Освобождаем память удаляемого узла
delete remove;

cout << "Элемент после индекса " << pos << " удалён." << endl;
}

// удаление по ключу
void removeByKey(Node*& head, char key) {
    // Если список пуст
    if (head == nullptr) {
        cout << "Список пуст, нечего удалять." << endl;
        return;
    }

    // Ищем узел с заданным ключом
    Node* target = findNodeByKey(head, key);

    // Если узел не найден
    if (target == nullptr) {
        cout << "Элемент с ключом '" << key << "' не найден в списке." << endl;
        return;
    }

    // Случай 1: удаляемый узел — первый в списке (head)
    if (target == head) {
        head = head->next; // Обновляем head на следующий узел
        if (head != nullptr) {
            head->prev = nullptr;
        }
        delete target; // Освобождаем память
        cout << "Элемент с ключом '" << key << "' удалён (был первым в списке)." << endl;
        return;
    }

    // Случай 2: удаляемый узел не первый
    if (target->prev != nullptr) {
        target->prev->next = target->next;
    }

    if (target->next != nullptr) {
        target->next->prev = target->prev;
    }

    delete target; // Освобождаем память
    cout << "Элемент с ключом '" << key << "' удалён." << endl;
}

// запись в файл
void saveListToFile(Node* head, const string& filename) {
    ofstream file(filename);
    if (!file.is_open()) {

```

```

        cout << "Ошибка: не удалось открыть файл '" << filename << "' для записи." <<
endl;
        return;
    }

    if (head == nullptr) {
        file << "Список пуст" << endl;
        cout << "Список записан в файл '" << filename << "'" << endl;
        file.close();
        return;
    }

    Node* current = head;
    while (current != nullptr) {
        file << current->data;
        if (current->next != nullptr) {
            file << " -> ";
        }
        current = current->next;
    }
    file << " -> nullptr" << endl;

    cout << "Список успешно записан в файл '" << filename << "'" << endl;
    file.close();
}

// чтение из файла
void loadListFromFile(Node*& head, const string& filename) {
    // Сначала уничтожаем текущий список
    destroyList(head);

    ifstream file(filename);
    if (!file.is_open()) {
        cout << "Ошибка: не удалось открыть файл '" << filename << "' для чтения." <<
endl;
        return;
    }

    string line;
    if (!getline(file, line)) {
        cout << "Файл пуст." << endl;
        file.close();
        return;
    }

    // Проверяем, пуст ли список
    if (line == "Список пуст") {
        cout << "Из файла загружен пустой список." << endl;
        file.close();
        return;
    }
}

```

```

// Удаляем "-> nullptr" в конце строки
size_t pos = line.find("-> nullptr");
if (pos != string::npos) {
    line = line.substr(0, pos);
}

// Разбираем строку на элементы
stringstream ss(line);
string element;

while (getline(ss, element, ' ')) {
    // Пропускаем стрелки "->"
    if (element == "->") continue;

    // Добавляем элемент в список
    pushBack(head, element[0]);
}

cout << "Список успешно загружен из файла " << filename << " " << endl;
printList(head);
file.close();
}

int main() {
    setlocale(LC_ALL, "ru");

    Node* head = nullptr;
    List list; // Инициализируем список с head и tail

    cout << "Введите 5 элементов списка: " << endl;
    for (int i = 0; i < 5; i++) {
        char a;
        cin >> a;
        addToBegin(head, a);
    }

    printList(head);

    char el;
    int pos;
    char key;
    string filename;
    int move = -1;
    while (move != 0) {
        cout << endl << R"(
Выберите действие:
0 - завершить программу
1 - добавление элемента в конец списка (head)
2 - добавление элемента в конец списка (head + tail)
3 - добавление элемента в начало списка
4 - добавление элемента после указанного индекса в списке
5 - добавление k элементов после элемента с заданным ключом

```

6 - удаление элемента из начала списка
7 - удаление элемента из конца списка
8 - удаление элемента после указанного индекса в списке
9 - удаление элемента с заданным ключом
10 - показ списка
11 - запись списка в файл
12 - уничтожение списка
13 - восстановление списка из файла
)" << endl;

cin >> move;

switch (move) {

case 0:

return 0;

case 1:

cout << "Введите элемент: ";

cin >> el;

pushBack(head, el);

printList(head);

break;

case 2:

cout << "Введите элемент: ";

cin >> el;

addToEndOptimized(list, el); // Добавляем в конец оптимизированного списка

copyListToHead(list, head); // Копируем обратно в head

printList(head);

break;

case 3:

cout << "Введите элемент: ";

cin >> el;

addToBegin(head, el);

printList(head);

break;

case 4:

char el;

cout << "Введите индекс, после которого вставить элемент: ";

cin >> pos;

cout << "Введите элемент для вставки: ";

cin >> el;

insertAfterIndex(head, el, pos);

printList(head);

break;

case 5:

int k;

cout << "Введите ключ (элемент), после которого добавить элементы: ";

cin >> key;

cout << "Введите количество элементов (k) для добавления: ";

```

    cin >> k;
    if (k <= 0) {
        cout << "Количество элементов должно быть положительным." << endl;
        break;
    }
    insertKElementsAfterKey(head, key, k);
    printList(head);
    break;

case 6:
    popFront(head);
    printList(head);
    break;

case 7:
    popBack(head);
    printList(head);
    break;

case 8:
    cout << "Введите индекс, после которого удалить элемент: ";
    cin >> pos;
    removeAfterIndex(head, pos);
    printList(head);
    break;

case 9:
    cout << "Введите ключ (значение) элемента для удаления: ";
    cin >> key;
    removeByKey(head, key);
    printList(head);
    break;

case 10:
    printList(head);
    break;

case 11:
    cout << "Введите имя файла для сохранения списка: ";
    cin >> filename;
    saveListToFile(head, filename);
    printList(head);
    break;

case 12:
    destroyList(head);
    printList(head);
    break;

case 13:
    cout << "Введите имя файла для загрузки списка: ";
    cin >> filename;

```

```

        loadListFromFile(head, filename);
        break;

    default:
        cout << "Неверный выбор. Попробуйте снова." << endl;
        break;
    }
}
}

```

Результат выполнения программы

```

Введите 5 элементов списка:
1 2 3 4 5
5 -> 4 -> 3 -> 2 -> 1 -> nullptr

```

Выберите действие:

```

0 - завершить программу
1 - добавление элемента в конец списка (head)
2 - добавление элемента в конец списка (head + tail)
3 - добавление элемента в начало списка
4 - добавление элемента после указанного индекса в списке
5 - добавление k элементов после элемента с заданным ключом
6 - удаление элемента из начала списка
7 - удаление элемента из конца списка
8 - удаление элемента после указанного индекса в списке
9 - удаление элемента с заданным ключом
10 - показ списка
11 - запись списка в файл
12 - уничтожение списка
13 - восстановление списка из файла

```

```

1
Введите элемент: 6
5 -> 4 -> 3 -> 2 -> 1 -> 6 -> nullptr

```

```

2
Введите элемент: 0
5 -> 4 -> 3 -> 2 -> 1 -> 6 -> 0 -> nullptr

```

```

3
Введите элемент: 7
7 -> 5 -> 4 -> 3 -> 2 -> 1 -> 6 -> 0 -> nullptr

```

4

Введите индекс, после которого вставить элемент: 2

Введите элемент для вставки: 1

7 -> 5 -> 4 -> 1 -> 3 -> 2 -> 1 -> 6 -> 0 -> nullptr

5

Введите ключ (элемент), после которого добавить элементы: 3

Введите количество элементов (k) для добавления: 2

Введите 1-й элемент из 2: 6

Введите 2-й элемент из 2: 7

Элементы успешно добавлены.

7 -> 5 -> 4 -> 1 -> 3 -> 6 -> 7 -> 2 -> 1 -> 6 -> 0 -> nullptr

6

Элемент из начала списка удалён.

5 -> 4 -> 1 -> 3 -> 6 -> 7 -> 2 -> 1 -> 6 -> 0 -> nullptr

7

Элемент из конца списка удалён.

5 -> 4 -> 1 -> 3 -> 6 -> 7 -> 2 -> 1 -> 6 -> nullptr

8

Введите индекс, после которого удалить элемент: 1

Элемент после индекса 1 удалён.

5 -> 4 -> 3 -> 6 -> 7 -> 2 -> 1 -> 6 -> nullptr

9

Введите ключ (значение) элемента для удаления: 5

Элемент с ключом '5' удалён (был первым в списке).

4 -> 3 -> 6 -> 7 -> 2 -> 1 -> 6 -> nullptr

10

4 -> 3 -> 6 -> 7 -> 2 -> 1 -> 6 -> nullptr

11

Введите имя файла для сохранения списка: 111

Список успешно записан в файл '111'

4 -> 3 -> 6 -> 7 -> 2 -> 1 -> 6 -> nullptr


```
111
Файл  Изменить  Просмотр
4 -> 3 -> 6 -> 7 -> 2 -> 1 -> 6 -> nullptr
```

12
Список пуст

13
Введите имя файла для загрузки списка: 111
Список успешно загружен из файла '111'
4 -> 3 -> 6 -> 7 -> 2 -> 1 -> 6 -> nullptr

0
C:\Users\elenalapteva\source\code\0 (0x0).
Чтобы автоматически закрыть

Стеки

Постановка задачи

Выберите действие:

- 0 - завершить программу
- 1 - добавление элемента
- 2 - удаление элемента
- 3 - показ стека
- 4 - запись стека в файл
- 5 - уничтожение стека
- 6 - восстановление стека из файла

Анализ задачи

Инициализация:

Устанавливает русскую локаль

Создаёт стек типа `char`

Инициализирует стек (`SetStack`)

Основной цикл:

Выводит меню с 6 опциями

Считывает выбор пользователя

Выполняет соответствующее действие через `switch`

Обработка команд:

Case 0: Завершение программы

Case 1 (Добавление):

Запрашивает символ у пользователя

Вызывает `Push`

Показывает обновлённый стек

Case 2 (Удаление):

Вызывает `Pop`

Если успешно — показывает удалённый символ и стек

Если стек пуст — выводит сообщение

Case 3 (Показ):

Вызывает `ShowStack`

Case 4 (Запись в файл):

Запрашивает имя файла

Вызывает `SaveStackToFile`

Case 5 (Уничтожение):

Вызывает `DestroyStack`

Case 6 (Восстановление):

Запрашивает имя файла

Вызывает `LoadStackFromFile`

Default:

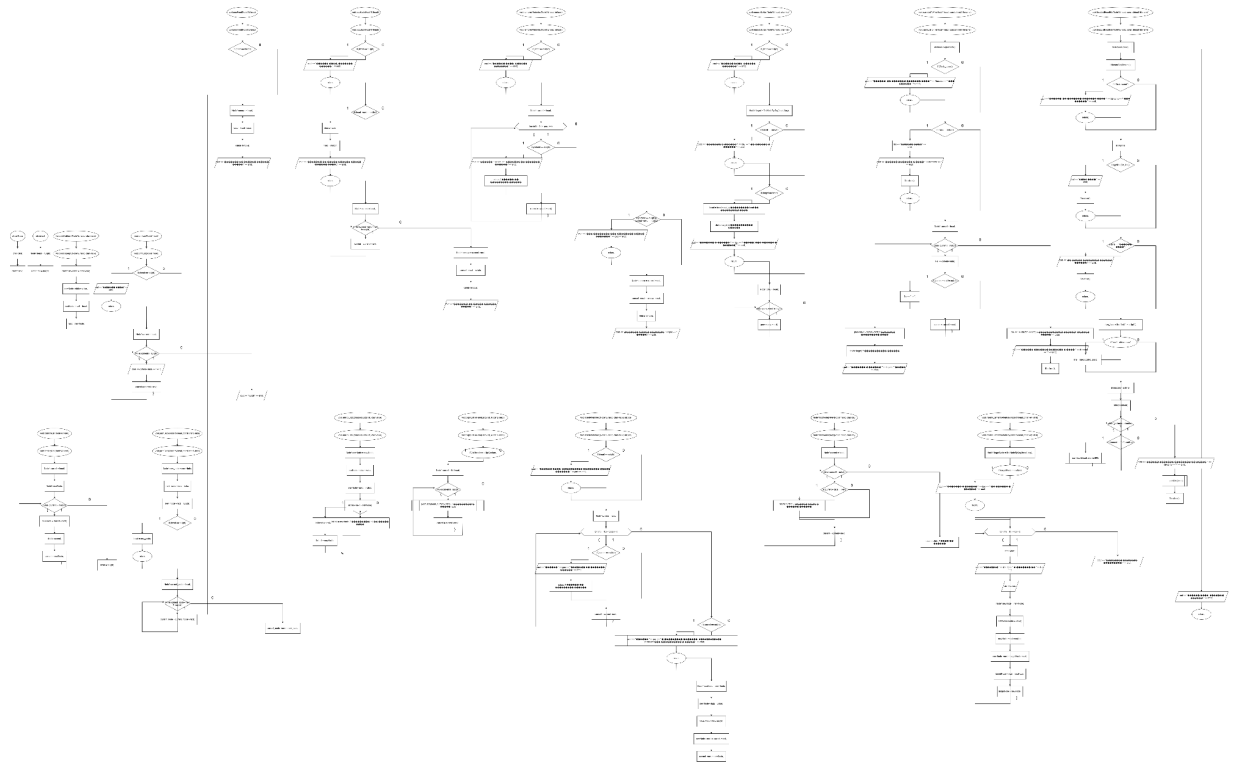
Выводит сообщение о неверном выборе

Этап 7: Завершение программы

При выборе 0 программа возвращает 0 и завершается

Автоматически освобождается память (деструкторы)

Блок-схема



Код

```
#include <iostream>
#include <fstream>
#include <string>
#include <sstream>
using namespace std;

// Структура "Узел"
template <class T>
struct Node
{
    T data;
    Node<T>* next_ptr;
};

// Структура Стекa
template <class T>
struct Stack
{
    Node<T>* head;
    int size;
};

// Функция создания узла
template <class T>
void SetNode(Node<T>*& node, T data = T(), Node<T>* next_ptr = nullptr)
{
    node = new Node<T>();
    node->next_ptr = next_ptr;
    node->data = data;
}

// Функция инициализации стека
template <class T>
```

```

void SetStack(Stack<T>& tmp)
{
    tmp.head = nullptr;
    tmp.size = 0;
}

// Функция добавления элемента (Push)
template <class T>
void Push(Stack<T>& tmp, const T& data)
{
    Node<T>* new_node;
    SetNode(new_node, data, tmp.head);
    tmp.head = new_node;
    tmp.size++;
}

// Функция удаления элемента (Pop)
template <class T>
bool Pop(Stack<T>& tmp, T& removed_data)
{
    if (tmp.head != nullptr)
    {
        Node<T>* delete_element = tmp.head;
        removed_data = delete_element->data;
        tmp.head = tmp.head->next_ptr;
        delete delete_element;
        tmp.size--;
        return true;
    }
    return false;
}

// Функция показа стека
template <class T>
void ShowStack(Stack<T>& tmp)
{
    if (tmp.head == nullptr)
    {
        cout << "Стек пуст" << endl;
        return;
    }

    Node<T>* current = tmp.head;
    cout << "Вершина стека -> ";
    while (current != nullptr)
    {
        cout << current->data;
        if (current->next_ptr != nullptr)
            cout << " -> ";
        current = current->next_ptr;
    }
    cout << " -> nullptr" << endl;
    cout << "Размер стека: " << tmp.size << endl;
}

// Функция уничтожения стека
template <class T>
void DestroyStack(Stack<T>& tmp)
{
    Node<T>* current = tmp.head;
    Node<T>* next_node;

    while (current != nullptr)

```

```

    {
        next_node = current->next_ptr;
        delete current;
        current = next_node;
    }
    tmp.head = nullptr;
    tmp.size = 0;
    cout << "Стек уничтожен" << endl;
}

// Функция записи стека в файл
template <class T>
void SaveStackToFile(Stack<T>& tmp, const string& filename)
{
    ofstream file(filename);
    if (!file.is_open())
    {
        cout << "Ошибка: не удалось открыть файл '" << filename << "' для записи." << endl;
        return;
    }

    if (tmp.head == nullptr)
    {
        file << "Стек пуст" << endl;
        cout << "Стек записан в файл '" << filename << "'" << endl;
        file.close();
        return;
    }

    Node<T>* current = tmp.head;
    while (current != nullptr)
    {
        file << current->data;
        if (current->next_ptr != nullptr)
            file << " -> ";
        current = current->next_ptr;
    }
    file << " -> nullptr" << endl;
    file << "Размер: " << tmp.size << endl;

    cout << "Стек успешно записан в файл '" << filename << "'" << endl;
    file.close();
}

// Функция восстановления стека из файла
template <class T>
void LoadStackFromFile(Stack<T>& tmp, const string& filename)
{
    DestroyStack(tmp);

    ifstream file(filename);
    if (!file.is_open())
    {
        cout << "Ошибка: не удалось открыть файл '" << filename << "' для чтения." << endl;
        return;
    }

    string line;
    if (!getline(file, line))
    {
        cout << "Файл пуст." << endl;
        file.close();
        return;
    }

```

```

    }

    if (line == "Стек пуст")
    {
        cout << "Из файла загружен пустой стек." << endl;
        file.close();
        return;
    }

    size_t pos = line.find("-> nullptr");
    if (pos != string::npos)
    {
        line = line.substr(0, pos);
    }

    // Для char читаем каждый символ
    for (size_t i = 0; i < line.length(); i++)
    {
        if (line[i] != ' ' && line[i] != '-' && line[i] != '>')
        {
            Push(tmp, line[i]);
        }
    }

    cout << "Стек успешно загружен из файла '" << filename << "'" << endl;
    ShowStack(tmp);
    file.close();
}

int main()
{
    setlocale(LC_ALL, "ru");

    Stack<char> stack;
    SetStack(stack);

    int move = -1;
    while (move != 0)
    {
        cout << endl << R"(
Выберите действие:
0 - завершить программу
1 - добавление элемента
2 - удаление элемента
3 - показ стека
4 - запись стека в файл
5 - уничтожение стека
6 - восстановление стека из файла
)" << endl;
        cin >> move;

        switch (move)
        {
            case 0:
                return 0;

            case 1:
            {
                char el;
                cout << "Введите символ: ";
                cin >> el;
                Push(stack, el);
                ShowStack(stack);
            }
        }
    }
}

```

```

        break;
    }

    case 2:
    {
        char removed;
        if (Pop(stack, removed))
        {
            cout << "Удалён символ: " << removed << " " << endl;
            ShowStack(stack);
        }
        else
        {
            cout << "Стек пуст, нечего удалять." << endl;
        }
        break;
    }

    case 3:
        ShowStack(stack);
        break;

    case 4:
    {
        string filename;
        cout << "Введите имя файла для сохранения стека: ";
        cin >> filename;
        SaveStackToFile(stack, filename);
        break;
    }

    case 5:
        DestroyStack(stack);
        break;

    case 6:
    {
        string filename;
        cout << "Введите имя файла для загрузки стека: ";
        cin >> filename;
        LoadStackFromFile(stack, filename);
        break;
    }

    default:
        cout << "Неверный выбор. Попробуйте снова." << endl;
        break;
    }
}

return 0;
}

```

Результат выполнения программы

Выберите действие:

- 0 – завершить программу
- 1 – добавление элемента
- 2 – удаление элемента
- 3 – показ стека
- 4 – запись стека в файл
- 5 – уничтожение стека
- 6 – восстановление стека из файла

1

Введите символ: k

Вершина стека -> k -> nullptr

Размер стека: 1

2

Удалён символ: 'f'

Вершина стека -> h -> k -> nullptr

Размер стека: 2

3

Вершина стека -> h -> k -> nullptr

Размер стека: 2

4

Введите имя файла для сохранения стека: 1234

Стек успешно записан в файл '1234'

5

Стек уничтожен

6

Введите имя файла для загрузки стека: 1234

Стек уничтожен

Стек успешно загружен из файла '1234'

Вершина стека -> k -> h -> nullptr

Размер стека: 2

0

C:\Users\elenalapteva\source\rep
одом 0 (0x0).

Чтобы автоматически закрывать ко

Очереди

Постановка задачи

Выберите действие:

- 0 - завершить программу
- 1 - добавление элемента
- 2 - удаление элемента
- 3 - показ стека
- 4 - запись стека в файл
- 5 - уничтожение стека
- 6 - восстановление стека из файла

Анализ задачи

Инициализация:

Устанавливает русскую локаль

Создаёт стек типа `char`

Инициализирует стек (`SetStack`)

Основной цикл:

Выводит меню с 6 опциями

Считывает выбор пользователя

Выполняет соответствующее действие через `switch`

Обработка команд:

Case 0: Завершение программы

Case 1 (Добавление):

Запрашивает символ у пользователя

Вызывает `Push`

Показывает обновлённый стек

Case 2 (Удаление):

Вызывает `Pop`

Если успешно — показывает удалённый символ и стек

Если стек пуст — выводит сообщение

Case 3 (Показ):

Вызывает `ShowStack`

Case 4 (Запись в файл):

Запрашивает имя файла

Вызывает `SaveStackToFile`

Case 5 (Уничтожение):

Вызывает `DestroyStack`

Case 6 (Восстановление):

Запрашивает имя файла

Вызывает `LoadStackFromFile`

Default:

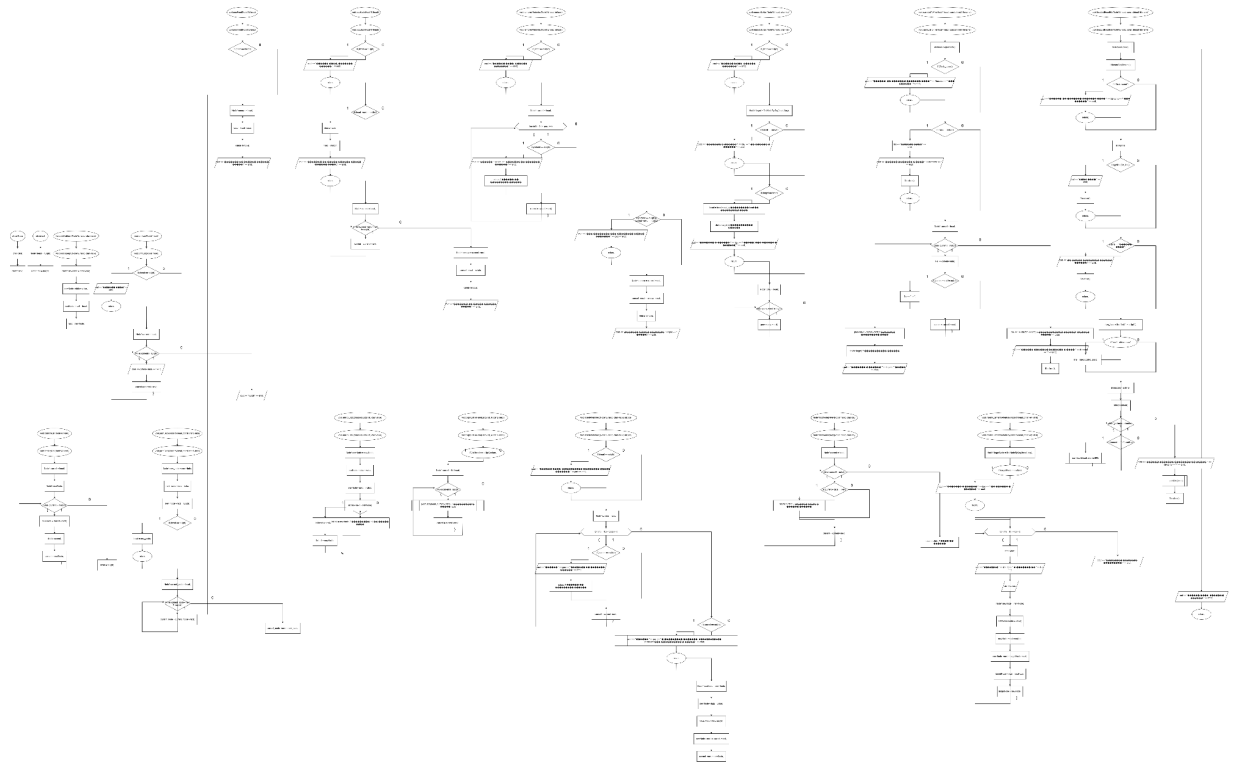
Выводит сообщение о неверном выборе

Этап 7: Завершение программы

При выборе 0 программа возвращает 0 и завершается

Автоматически освобождается память (деструкторы)

Блок-схема



Код

```
#include <iostream>
#include <fstream>
#include <string>
#include <sstream>
using namespace std;

// Структура "Узел" (остаётся без изменений)
template <class T>
struct Node
{
    T data;
    Node<T>* next_ptr;
};

// Структура Очереди
template <class T>
struct Queue
{
    Node<T>* head; // начало очереди (отсюда удаляем)
    Node<T>* tail; // конец очереди (сюда добавляем)
    int size;
};

// Функция создания узла
template <class T>
void SetNode(Node<T>*& node, T data = T(), Node<T>* next_ptr = nullptr)
{
    node = new Node<T>();
    node->next_ptr = next_ptr;
    node->data = data;
}

// Функция инициализации очереди
```

```

template <class T>
void SetQueue(Queue<T>& tmp)
{
    tmp.head = nullptr;
    tmp.tail = nullptr;
    tmp.size = 0;
}

// Функция добавления элемента в очередь (Enqueue) - в конец!
template <class T>
void Enqueue(Queue<T>& tmp, const T& data)
{
    Node<T>* new_node;
    SetNode<T>(new_node, data, nullptr); // Явно указываем <T>

    if (tmp.tail == nullptr)
    {
        tmp.head = tmp.tail = new_node;
    }
    else
    {
        tmp.tail->next_ptr = new_node;
        tmp.tail = new_node;
    }
    tmp.size++;
}

// Функция удаления элемента из очереди (Dequeue) - из начала!
template <class T>
bool Dequeue(Queue<T>& tmp, T& removed_data)
{
    if (tmp.head != nullptr)
    {
        Node<T>* delete_element = tmp.head;
        removed_data = delete_element->data;
        tmp.head = tmp.head->next_ptr;

        // Если очередь стала пустой, обновляем и tail
        if (tmp.head == nullptr)
            tmp.tail = nullptr;

        delete delete_element;
        tmp.size--;
        return true;
    }
    return false;
}

// Функция показа очереди
template <class T>
void ShowQueue(Queue<T>& tmp)
{
    if (tmp.head == nullptr)
    {
        cout << "Очередь пуста" << endl;
        return;
    }

    Node<T>* current = tmp.head;
    cout << "Начало очереди -> ";
    while (current != nullptr)
    {
        cout << current->data;
    }
}

```

```

        if (current->next_ptr != nullptr)
            cout << "-> ";
        current = current->next_ptr;
    }
    cout << "-> nullptr" << endl;
    cout << "Размер очереди: " << tmp.size << endl;
}

```

```

// Функция уничтожения очереди
template <class T>
void DestroyQueue(Queue<T>& tmp)
{
    Node<T>* current = tmp.head;
    Node<T>* next_node;

    while (current != nullptr)
    {
        next_node = current->next_ptr;
        delete current;
        current = next_node;
    }
    tmp.head = nullptr;
    tmp.tail = nullptr;
    tmp.size = 0;
    cout << "Очередь уничтожена" << endl;
}

```

```

// Функция записи очереди в файл
template <class T>
void SaveQueueToFile(Queue<T>& tmp, const string& filename)
{
    ofstream file(filename);
    if (!file.is_open())
    {
        cout << "Ошибка: не удалось открыть файл '" << filename << "' для записи." << endl;
        return;
    }

```

```

    if (tmp.head == nullptr)
    {
        file << "Очередь пуста" << endl;
        cout << "Очередь записана в файл '" << filename << "'" << endl;
        file.close();
        return;
    }

```

```

    Node<T>* current = tmp.head;
    while (current != nullptr)
    {
        file << current->data;
        if (current->next_ptr != nullptr)
            file << "-> ";
        current = current->next_ptr;
    }
    file << "-> nullptr" << endl;
    file << "Размер: " << tmp.size << endl;

```

```

    cout << "Очередь успешно записана в файл '" << filename << "'" << endl;
    file.close();
}

```

```

// Функция восстановления очереди из файла
template <class T>

```

```

void LoadQueueFromFile(Queue<T>& tmp, const string& filename)
{
    DestroyQueue(tmp);

    ifstream file(filename);
    if (!file.is_open())
    {
        cout << "Ошибка: не удалось открыть файл " << filename << " для чтения." << endl;
        return;
    }

    string line;
    if (!getline(file, line))
    {
        cout << "Файл пуст." << endl;
        file.close();
        return;
    }

    if (line == "Очередь пуста")
    {
        cout << "Из файла загружена пустая очередь." << endl;
        file.close();
        return;
    }

    size_t pos = line.find("-> nullptr");
    if (pos != string::npos)
    {
        line = line.substr(0, pos);
    }

    // Для char читаем каждый символ (аналогично твоему коду)
    for (size_t i = 0; i < line.length(); i++)
    {
        if (line[i] != ' ' && line[i] != '-' && line[i] != '>')
        {
            Enqueue(tmp, line[i]);
        }
    }

    cout << "Очередь успешно загружена из файла " << filename << "" << endl;
    ShowQueue(tmp);
    file.close();
}

int main()
{
    setlocale(LC_ALL, "ru");

    Queue<char> queue;
    SetQueue(queue);

    int move = -1;
    while (move != 0)
    {
        cout << endl << R"(
Выберите действие:
0 - завершить программу
1 - добавление элемента (в конец очереди)
2 - удаление элемента (из начала очереди)
3 - показ очереди
4 - запись очереди в файл

```

5 - уничтожение очереди

6 - восстановление очереди из файла

)" << endl;

cin >> move;

switch (move)

{

case 0:

return 0;

case 1:

{

char el;

cout << "Введите символ: ";

cin >> el;

Enqueue(queue, el);

ShowQueue(queue);

break;

}

case 2:

{

char removed;

if (Dequeue(queue, removed))

{

cout << "Удалён символ: " << removed << " " << endl;

ShowQueue(queue);

}

else

{

cout << "Очередь пуста, нечего удалять." << endl;

}

break;

}

case 3:

ShowQueue(queue);

break;

case 4:

{

string filename;

cout << "Введите имя файла для сохранения очереди: ";

cin >> filename;

SaveQueueToFile(queue, filename);

break;

}

case 5:

DestroyQueue(queue);

break;

case 6:

{

string filename;

cout << "Введите имя файла для загрузки очереди: ";

cin >> filename;

LoadQueueFromFile(queue, filename);

break;

}

default:

cout << "Неверный выбор. Попробуйте снова." << endl;

```
        break;
    }
}

return 0;
}
```

Результат выполнения программы

```
Выберите действие:
0 - завершить программу
1 - добавление элемента (в конец очереди)
2 - удаление элемента (из начала очереди)
3 - показ очереди
4 - запись очереди в файл
5 - уничтожение очереди
6 - восстановление очереди из файла
```

```
1
Введите символ: 2
Начало очереди -> 2 -> nullptr
Размер очереди: 1
```

```
2
Удалён символ: '2'
Начало очереди -> 3 -> 4 -> nullptr
Размер очереди: 2
```

```
3
Начало очереди -> 3 -> 4 -> nullptr
Размер очереди: 2
```

```
4
Введите имя файла для сохранения очереди: 1
Очередь успешно записана в файл '1'
```

```
5
Очередь уничтожена
```


6

Введите имя файла для загрузки очереди: 1

Очередь уничтожена

Очередь успешно загружена из файла '1'

Начало очереди -> 3 -> 4 -> nullptr

Размер очереди: 2

0

C:\Users\elenalapteva\source\repos\test

одом 0 (0x0).

Чтобы автоматически закрывать консоль

АВТОМАТИЧЕСКИ ЗАКРЫТЬ КОНСОЛЬ ПРИ ОСТА